

CS683: Advanced Computer Architecture

Programming Assignment 1: Dhurandhar Programming

TEAM: Latency Hunters

Autumn 2026

Table 1: Hardware Configuration Summary

Component	Specification
Processor	Intel Core Ultra 9 275HX (Max. freq. 6.5 GHz)
Microarchitecture	Arrow Lake-HX
Cores/Threads	24 Cores, 24 Threads
L1d Cache	768 KiB total (20 instances)
L1i Cache	1.3 MiB total (20 instances)
L2 Cache	40 MiB total (12 instances)
L3 Cache	36 MiB shared (1 instance)
ISA Extensions	AVX, AVX2, AVX-VNNI, FMA, BMI1, BMI2, GFNI, VAES
Memory	96 GB DDR5-5600 (2 × 48 GB SODIMM)

1 Single Instruction Multiple Data (SIMD) Instructions

We have implemented a SIMD version of the matrix multiplication kernel that uses AVX2 instructions and 8-register tiling. The instruction count, time taken, speedup (compared to naive implementation) have all been tabulated in 2 and 3. All parts of this experiment were done with hardware prefetchers turned off.

1.1 Analyze the Impact of SIMD Instruction Width

In our experiments we have tested using two widths of SIMD instructions:

1. 128-bit SIMD
2. 256-bit SIMD

1.2 Analyze the Impact of Matrix Sizes

In our experiments we have tested with five different matrix sizes (written in form of (M, N, K) where the matrices multiplied are of the order $M \times K$ and $K \times N$):

1. (256, 256, 256)
2. (512, 512, 512)
3. (1024, 1024, 1024)
4. (2048, 2048, 2048)
5. (4096, 4096, 4096)

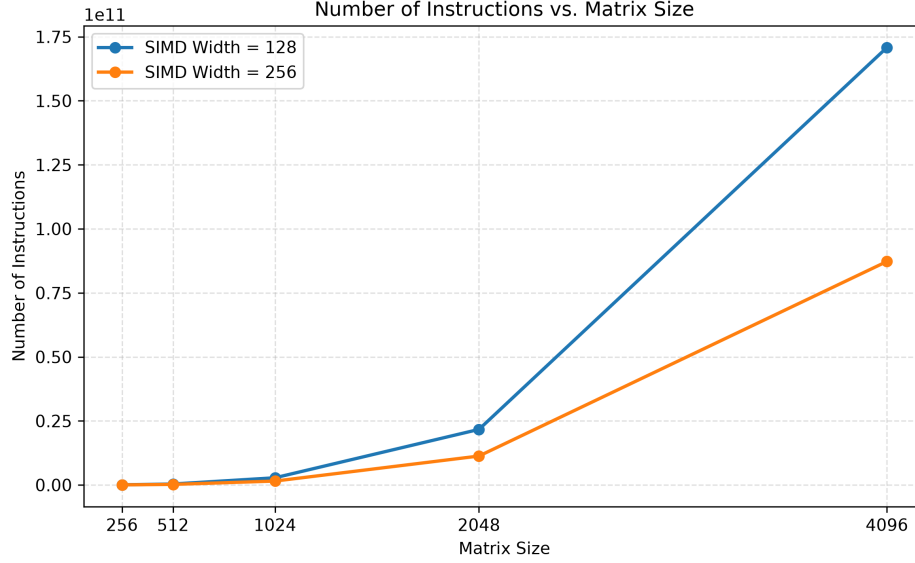


Figure 1: Instruction counts for different SIMD widths and matrix sizes

1.3 Analyze Instruction Count

The impact of different SIMD widths in the instruction count has been tabulated in Table 2 and depicted in Figure 1. We have observed that:

1. Doubling SIMD width approximately halves the instruction count.
2. Using SIMD heavily decreases (by more than 90%) the number of instructions compared to naive multiplication approach.

Table 2: Instruction counts for different SIMD widths and matrix sizes

Matrix Size	SIMD Width	Instr. (SIMD)	Instr. (Naive)	Instr. Reduction
4096	256	87,247,873,772	2,479,109,969,945	96.48%
4096	128	170,781,106,197	2,479,109,969,945	93.11%
2048	256	11,291,146,926	310,277,368,916	96.36%
2048	128	21,692,426,812	310,277,368,916	93.00%
1024	256	1,517,409,263	38,880,032,865	96.10%
1024	128	2,806,422,582	38,880,032,865	92.78%
512	256	217,779,235	4,888,320,531	95.54%
512	128	376,935,911	4,888,320,531	92.29%
256	256	36,340,723	620,376,932	94.14%
256	128	55,625,683	620,376,932	91.03%

1.4 Collection Execution Time and Compute Speedup

The speedups for different SIMD widths and matrix sizes have been tabulated in Table 3 and depicted in Figure 2.

We have observed that:

1. Increase in SIMD-width increases the speedup. However, the speedup is not doubled on doubling SIMD width. Maximum speed is achieved with 256-bit width SIMD.

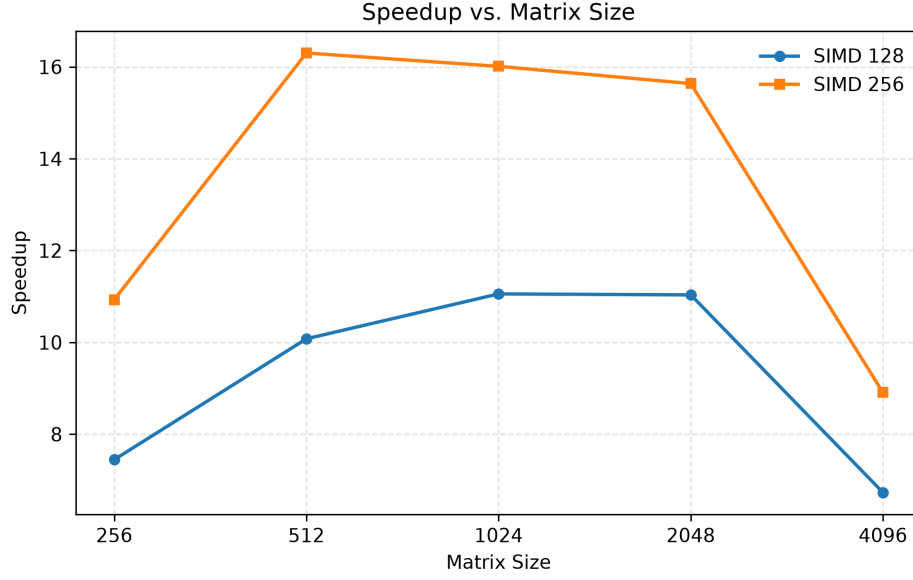


Figure 2: Speedup vs Matrix Size

2. Increase in matrix size at first increases the speedup, but it starts decreasing after matrix size becomes larger than the size of the cache hierarchy.

Table 3: Comparison of SIMD and naive execution times and speedup for different matrix sizes and SIMD widths.

Matrix Size	SIMD Width	Time (SIMD) (s)	Time (Naive) (s)	Speedup
4096	256	32.2531	287.3078	8.91
4096	128	42.6813	287.3078	6.73
2048	256	1.2917	20.1964	15.64
2048	128	1.8303	20.1964	11.03
1024	256	0.1515	2.4263	16.01
1024	128	0.2195	2.4263	11.06
512	256	0.0174	0.2828	16.30
512	128	0.0281	0.2828	10.08
256	256	0.0030	0.0331	10.93
256	128	0.0045	0.0331	7.45

1.5 Questions

1.5.1 What trends do you observe in speedup for different combinations of matrix sizes and SIMD widths?

As we go for SIMD 256 we got more speed up than SIMD 128. roughly giving 30–50% greater speedup depending on matrix size. The best performance is observed for medium-sized matrices (particularly 512–1024), while extremely large and very small matrices show reduced speedup.

1.5.2 For which SIMD width do you achieve the maximum speedup?

The best Speed up between the two SIMD width is due to 256-bit width. As it is doing 2× computation (than 128-bit SIMD) in every cycle but not all the instructions are vectorized like loops, jump instruction.

It follows the Amdahl's Law

$$\begin{aligned} T_{\text{new}} &= (1 - \alpha)T_{\text{old}} + \frac{\alpha T_{\text{old}}}{k} \\ &= T_{\text{old}} \left[(1 - \alpha) + \frac{\alpha}{k} \right] \end{aligned}$$

From this, we can compute the speedup $S = \frac{T_{\text{old}}}{T_{\text{new}}}$ as

$$S = \frac{1}{(1 - \alpha) + \frac{\alpha}{k}}$$

so here $k = 2$ but α is the part having vectorization. and it is not 1. So, SIMD 256 is $1.3\times$ to $1.5\times$ better than SIMD 128.

2 Software Prefetching

In this experiment we have added software prefetch requests and cache blocking to our SIMD implementation. We have tested for different prefetch distances, cache fill levels, and block sizes. These tests were also done with hardware prefetchers turned off.

2.1 Analyze the Impact of Matrix Size

In our experiments we have tested with four different matrix sizes (written in form of (M, N, K) where the matrices multiplied are of the order $M \times K$ and $K \times N$):

1. (512, 512, 512)
2. (1024, 1024, 1024)
3. (2048, 2048, 2048)
4. (4096, 4096, 4096)

A comparison of the best speedups achieved for each matrix size has been given in Table 4. We have observed that:

1. this approach gives less performance then basic SIMD for matrix sizes that fit inside the cache hierarchy.
2. this approach gives higher performance than basic SIMD for matrix sizes that are beyond the size of the cache hierarchy.

Table 4: Speedup achieved for different matrix sizes.

Matrix Size	Best Speedup
512	14.75
1024	14.99
2048	14.90
4096	22.28

2.2 Analyze CPU Metrics

We have analyzed the L1D MPKI (Misses Per Kilo Instructions), L2 MPKI, and L3 MPKI for:

1. Prefetch Distances: 32, 64, 128, 256, 512
2. Cache Fill Levels: T0 (`_MM_HINT_T0`), T1 (`_MM_HINT_T1`) and T3 (`_MM_HINT_T3`), used with `_mm_prefetch`
3. All the four matrix sizes mentioned in 2.1

We have observed the following:

1. Cache prefetch distance is increased within the same cache fill level have very less effect in the speed up for small matrices like 512, 1024, 2048 but significant change can be observed in the large matrix like 4096.
2. For a given matrix size changing the cache fill level from T0 to T1 have significant change in the speed up but it become more drastic when going to T2 from T0/T1. For T2 the speed up becomes actually slow down as prefetching to LLC actually does not hide memory latency much.

Table 5: Effect of prefetch distance and cache fill level on cache miss rates for different matrix sizes.

Matrix Size	Prefetch Distance	CLL	L1D MPKI	L2 MPKI	LLC MPKI	Speedup
512	32	T0	0.8339	0.0376	0.0094	13.0152
512	64	T0	1.2444	0.0426	0.0076	13.0005
512	128	T0	2.1240	0.0442	0.0078	12.9090
512	256	T0	3.8651	0.0701	0.0102	13.5931
512	512	T0	2.3943	0.0333	0.0120	12.6204
512	32	T1	11.0638	0.0572	0.0192	11.1348
512	64	T1	11.6861	0.0657	0.0183	11.2003
512	128	T1	10.8850	0.0631	0.0169	11.1819
512	256	T1	12.9266	0.0637	0.0136	11.2274
512	512	T1	12.6992	0.0481	0.0178	10.9172
512	32	T2	11.3398	0.0532	0.0133	11.7842
512	64	T2	11.1828	0.0652	0.0172	11.1946
512	128	T2	11.8340	0.0645	0.0166	11.2564
512	256	T2	12.1637	0.0603	0.0135	11.3164
512	512	T2	13.0476	0.0443	0.0175	11.2999
1024	32	T0	0.9425	0.0535	0.0063	14.3732
1024	64	T0	1.5035	0.0593	0.0064	14.3737
1024	128	T0	2.6538	0.0636	0.0084	14.3921
1024	256	T0	4.9728	0.0737	0.0128	14.2607
1024	512	T0	9.6164	0.0777	0.0273	14.2777
1024	32	T1	32.3774	0.0504	0.0403	12.2848
1024	64	T1	32.3702	0.0559	0.0357	12.3245
1024	128	T1	32.4932	0.0589	0.0322	12.4920
1024	256	T1	32.5120	0.0591	0.0219	12.5862
1024	512	T1	32.5693	0.0602	0.0200	12.4508
1024	32	T2	32.4745	0.0494	0.0408	1.4410
1024	64	T2	32.5601	0.0560	0.0338	1.4503
1024	128	T2	32.5962	0.0585	0.0314	1.4652
1024	256	T2	32.6151	0.0593	0.0205	1.4503
1024	512	T2	32.6463	0.0599	0.0163	1.4786
2048	32	T0	0.7099	0.0382	0.0078	14.3533
2048	64	T0	0.9674	0.0438	0.0094	14.2889
2048	128	T0	1.5238	0.0470	0.0109	14.5442
2048	256	T0	2.7220	0.0525	0.0142	14.4562
2048	512	T0	5.0589	0.0530	0.0430	14.1367
2048	32	T1	35.1329	0.0361	0.1046	11.9020
2048	64	T1	35.1117	0.0399	0.1022	12.0606
2048	128	T1	35.0947	0.0418	0.0978	12.0910
2048	256	T1	35.1442	0.0443	0.0835	12.0687
2048	512	T1	35.0832	0.0435	0.0331	12.1308

Matrix Size	Prefetch Distance	CLL	L1D MPKI	L2 MPKI	LLC MPKI	Speedup
2048	32	T2	35.1263	0.0362	0.0934	0.1674
2048	64	T2	35.1011	0.0403	0.0922	0.1697
2048	128	T2	35.0822	0.0417	0.0802	0.1702
2048	256	T2	35.1253	0.0421	0.0578	0.1705
2048	512	T2	35.0801	0.0438	0.0294	0.1704
4096	32	T0	0.2478	0.0399	0.0147	18.7409
4096	64	T0	0.3880	0.0447	0.0204	19.3383
4096	128	T0	0.6892	0.0453	0.0310	20.1854
4096	256	T0	1.2994	0.0462	0.0528	21.2886
4096	512	T0	2.5227	0.0554	0.1089	21.8462
4096	32	T1	34.8828	0.0419	1.1883	14.0433
4096	64	T1	34.8156	0.0489	1.0728	14.2714
4096	128	T1	34.8224	0.0503	0.8887	14.7070
4096	256	T1	34.8587	0.0541	0.4083	15.1164
4096	512	T1	34.8722	0.0635	0.1502	15.2024
4096	32	T2	34.8895	0.0420	1.1728	0.0138
4096	64	T2	34.8010	0.0477	1.0463	0.0141
4096	128	T2	34.8245	0.0505	0.8781	0.0145
4096	256	T2	34.8579	0.0538	0.3955	0.0149
4096	512	T2	34.8655	0.0599	0.1433	0.0150

2.3 Collect Execution Time and Compute Speedup

Execution times have been recorded for different prefetching parameters, and speedups have been calculated which are shown in Table 5.

2.4 Identify the Optimal Parameters

Based on our observations, we have found that:

1. Cache fill level T0 (`_MM_HINT_T0`) works the best. With all other cache fill levels, speedup decreases quite a bit. This is expected because T0 means highest locality is expected for the data prefetched, and thus the data is put in L1 cache (L1D MPKI is very high for T1 and T2), and because of the fact that we are prefetching in the inner-most loop of our logic, which means that we would expect to use the data in one of the next iterations of the loop. This can be seen in the plots of Figure 3.
2. After trying out various prefetch distances, we have found out that generally higher values like 256 and 512 outperform the others. 512 has performed best for larger matrix sizes like 4096×4096 , and otherwise 256 has performed better than the remaining. This can be seen in the plots of Figure 4. Hence we would call 256 the best value for prefetch distance.

2.5 Study the Effect of Hardware Prefetchers

We have observed that using hardware prefetchers slightly improve the performance over what is achieved using no prefetching at all. However, our software prefetching strategy does not provide better results than no prefetching, in matrix sizes smaller than the cache hierarchy size. This is because the performance gained from hiding memory latency could not compensate for the performance lost due to extra prefetch instructions, as whole workload can fit inside the cache hierarchy. It should however be noted that, for large matrix sizes like 4096×4096 , software prefetching did provide subtle improvements over no prefetching albeit still less than what is achieved using hardware prefetching.

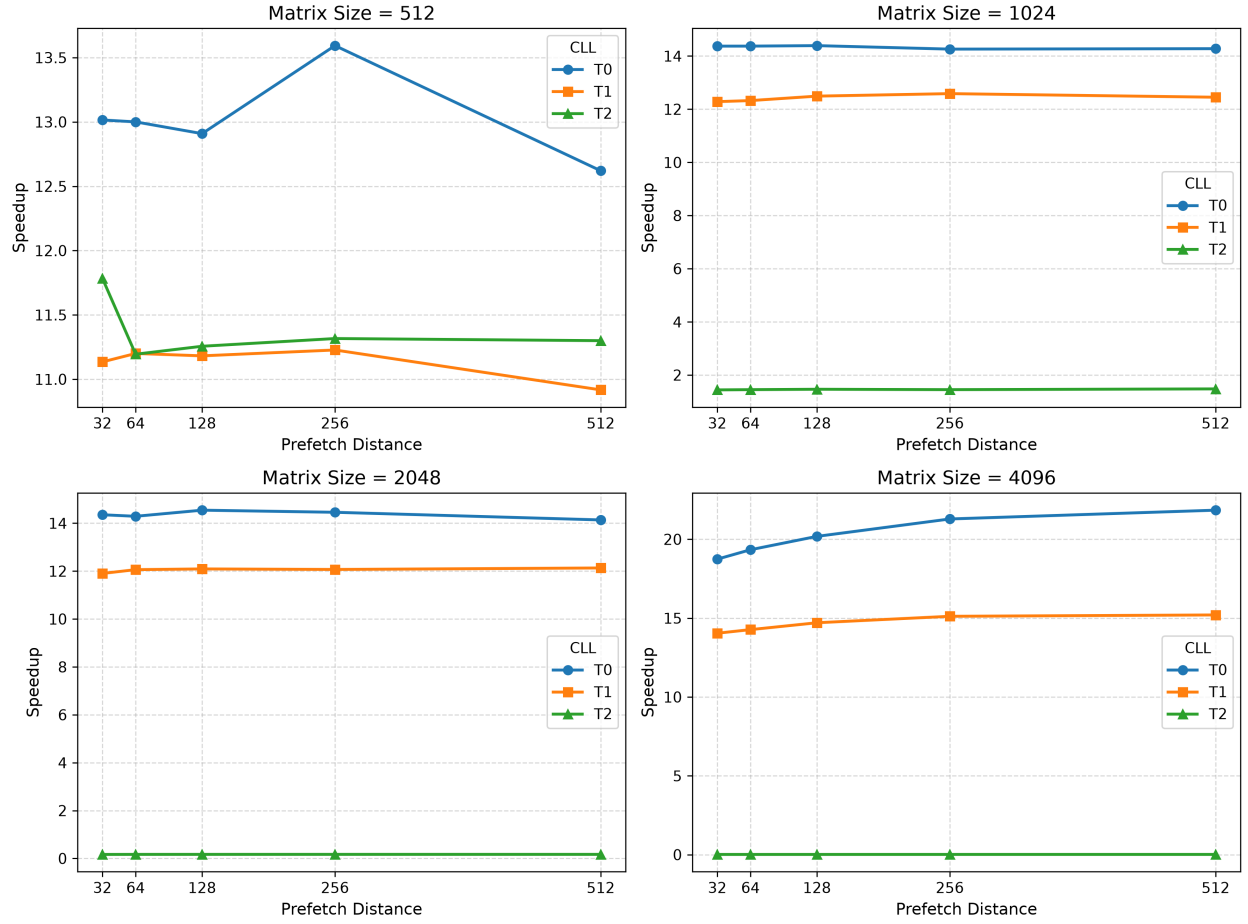


Figure 3: Variation of speedup with prefetch distance

HW Prefetcher	Matrix Size	L1D MPKI	L2 MPKI	LLC MPKI	Speedup
YES	4096	11.8977	29.6933	0.2359	24.1481
YES	2048	1.1502	25.0926	0.0363	17.5595
YES	1024	0.1426	22.1984	0.0250	17.0098
YES	512	0.0173	8.7045	0.0177	16.3409
NO	4096	13.6425	48.4150	1.7481	21.0598
NO	2048	1.2415	47.9243	0.1633	16.2682
NO	1024	0.1481	44.9795	0.0631	16.3876
NO	512	0.0208	15.1070	0.0426	13.6114

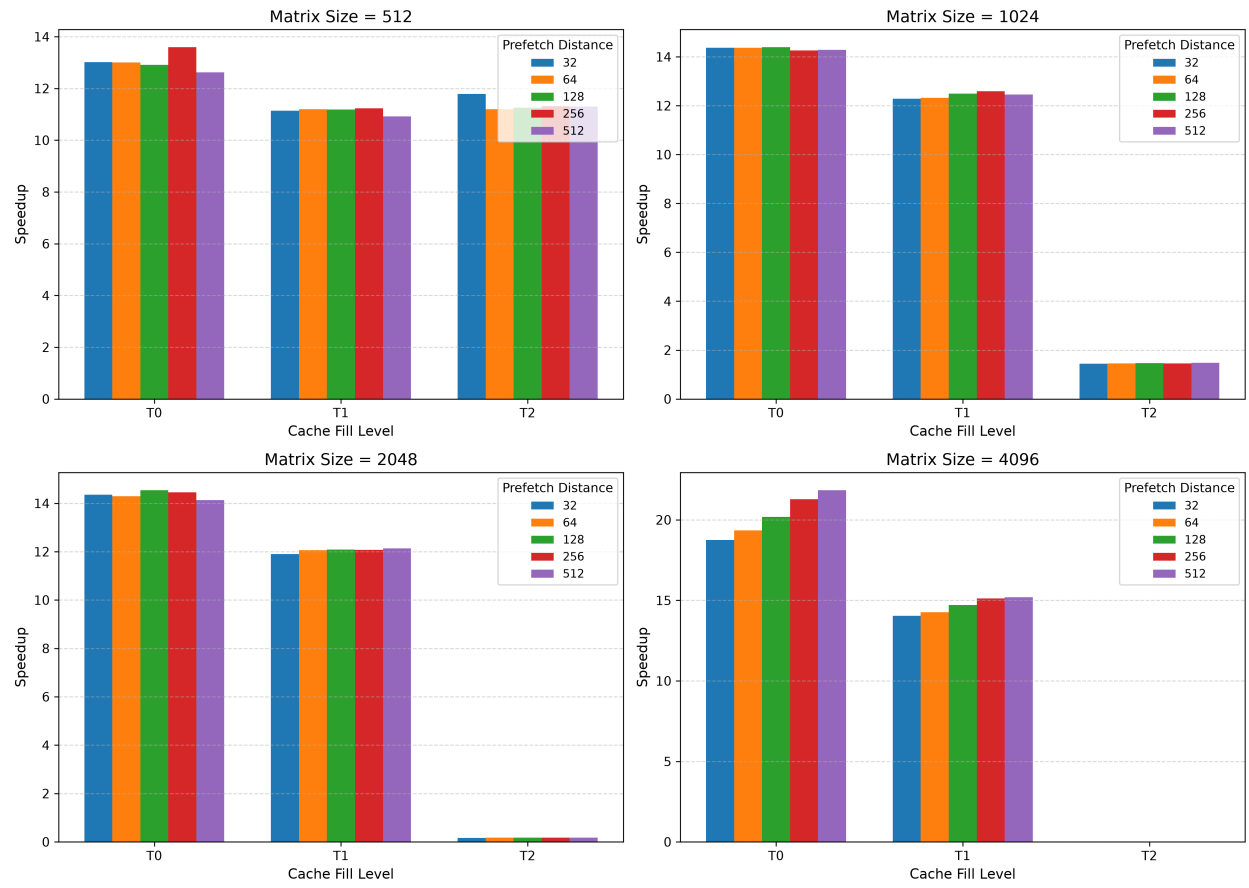


Figure 4: Variation of speedup with cache fill level

2.6 Questions

2.6.1 What trend do you observe in speedup with different matrix sizes?

We have observed that with increase in matrix sizes, the speedup remains more or less the same until the matrix size starts to increase beyond the size of the cache hierarchy. We have tabulated matrix sizes against speedups observed in Table 4.

2.6.2 What is the best prefetch distance?

We have found out 256 to be the best prefetch distance.

For sharp observation it depends on the size of the matrix and cache fill levels. As per our data for T0 and matrix sizes 1024×1024 , 2048×2048 the best prefetch distance is 128, for 512×512 it is 256 and for 4096×4096 it is 512. For T1 and matrix 512×512 , 1024×1024 speed up is max at prefetch distance 256 but for matrix 2048×2048 , 4096×4096 it is 512.

But in each case prefetch distance 256 is either has the best speed up or it's speed up is within 10% range of the best speed up for the given matrix and given cache fill level.

2.6.3 At what cache fill level do you achieve the maximum speedup?

We have achieved the maximum speedup at cache fill level T0.

T0 means to put the data in the closest level to cpu as highest locality is expected. So, don't need to travel any of the hierarchies for major portion of the data if data is prefetched before cpu actually request the data directly from L1 not going beyond it.

3 Optimized Approach

In the optimized approach, we have used 12 register tiled SIMD, with cache blocking with block size 96, loop reordering and loop unrolling.

3.1 Speedup and CPU Metrics

Various metrics regarding the approach are tabulated in Table 7 and Table 8.

Table 7: Speedup across different matrix sizes

Matrix Size	Speedup
4096	28.6741
2048	20.5645
1024	20.5090
512	17.6454

Table 8: CPU metrics for different matrix sizes

Matrix Size	Instructions	L1D MPKI	L2 MPKI	LLC MPKI
4096	85198950536	80.3852	0.4932	2.7156
2048	11095091952	77.2872	0.2192	0.2015
1024	1497595858	69.4179	0.2018	0.0839
512	218130176	27.5721	0.1130	0.0367

3.2 Software Prefetching

During testing, we have observed that software prefetching degrades the performance of the algorithm (in terms of speedup), and hence we dropped software prefetching in the optimized version. The metrics with and without software prefetching are presented in Table 9.

Table 9: Effect of software prefetching on optimized version

SW Prefetch	Matrix Size	L1D MPKI	L2 MPKI	LLC MPKI	Speedup
NO	4096	80.3852	0.4932	2.7156	28.6741
NO	2048	77.2872	0.2192	0.2015	20.5645
NO	1024	69.4179	0.2018	0.0839	20.5090
NO	512	27.5721	0.1130	0.0367	17.6454
YES	4096	2.6872	0.0773	0.1045	26.4967
YES	2048	5.2803	0.0779	0.0265	16.9951
YES	1024	9.7808	0.1053	0.0232	16.6775
YES	512	8.3947	0.1217	0.0202	14.4735

3.3 Comparison of Performance Among Three Approaches

Comparison of performance among the three approaches has been tabulated in Table 10 and plotted in Figure 5.

Table 10: Speedup comparison of different optimization approaches

Approach	Matrix Size	Speedup
SIMD	4096	8.9079
SIMD	2048	15.6355
SIMD	1024	16.0109
SIMD	512	16.3010
PREFETCH	4096	22.2790
PREFETCH	2048	14.9046
PREFETCH	1024	14.9905
PREFETCH	512	14.7495
OPTIMIZED	4096	28.6741
OPTIMIZED	2048	20.5645
OPTIMIZED	1024	20.5090
OPTIMIZED	512	17.6454

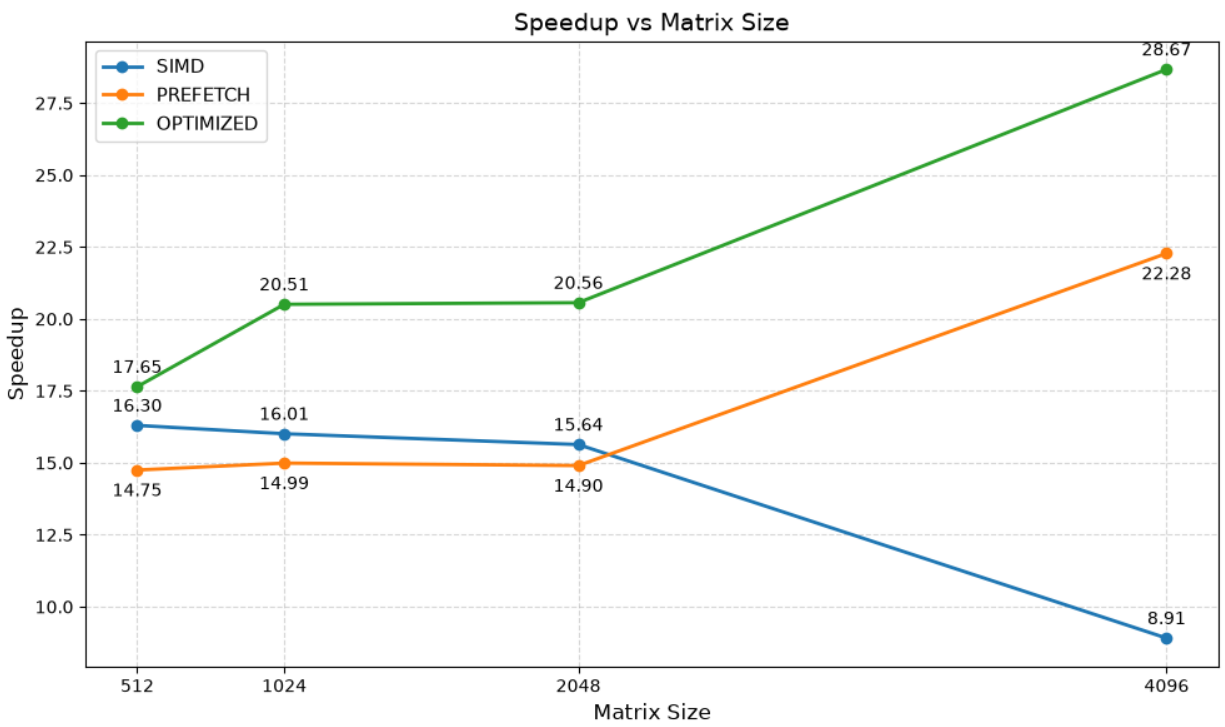


Figure 5: Performance comparison of the three approaches